

5. 템플릿 리터럴

1. 템플릿 리터럴

내장된 표현식을 허용하는 문자열 리터럴입니다. 여러 줄로 이뤄진 문자열과 문자 보간기능을 사용할 수 있습니다.

템플릿 리터럴은 이중 따옴표 나 작은 따옴표 대신 백틱(` `)을 사용합니다. 템플릿 리터럴은 또한 플레이스 홀더를 이용하여 표현식을 넣을 수 있는데, 이는 \$와 중괄호(\$ {expression}) 로 표기할 수 있습니다.

```
{% code lineNumbers="true" %}
```

```
<body>
  <Label id="myAgeTemplateLiterals" ></Label></br>
  <Label id="myAge" ></Label></br>
</body>
<script>

  const customer = {
    name : "홍길동",
    year : 1982,
    month : 8,
    day : 12
  }

  const fnAgeMsgTemplateLiterals = (customer) => {
    return `${customer.name}님의 나이는
      ${fnMakeAge(customer.year,
        customer.month,
        customer.day)}
      입니다.`;
  }

  const fnAgeMsg = (customer) => {
    return customer.name + '님의 나이는 '
      + fnMakeAge(customer.year,
        customer.month,
        customer.day)
      + ' 입니다.';
  }

  const fnMakeAge = (year, month, day) => {
    const today = new Date();
    const birthDate = new Date(year, month, day);
    return today.getFullYear()
      - birthDate.getFullYear()
      + 1;
  }

  document.querySelector("#myAgeTemplateLiterals")
```

```
        .innerHTML = fnAgeMsgTemplateLiterals(customer);  
  
document.querySelector("#myAge")  
    .innerHTML = fnAgeMsg(customer);
```

{% endcode %}

- 템플릿 리터럴은 가독성이 좋아집니다.
 - fnAgeMsgTemplateLiterals 함수는 템플릿 리터럴을 사용하였고 fnMakeAge함수는 + 연산을 사용하여 작성한 코드 입니다. 만약 문자열에 추가할 문장이 많으면 가독성 측면에서 템플릿 리터럴을 사용한 것 보다 + 연산을 사용한 것이 가독성이 낮을 것 입니다. \
- 개발자 도구에서도 템플릿 리터럴을 사용하여 표현식을 문자열과 함께 조합하여 사용할 수 있습니다.

